# GATE CS — Databases (DBMS)

Concise study notes for GATE 2027 · built from the 2016–2026 papers in this repo

## Where the marks come from (average marks per paper, 2016–2026)

| Topic | Marks / paper | Priority |
|---|---|---|
| SQL, relational algebra & calculus | 2.3 | Tier 2 |
| Normalization & FDs | 1.8 | Tier 2 |
| Transactions & concurrency control | 1.5 | Tier 2 |
| Indexing, B/B+ trees, file organization | 0.9 | Tier 3 |
| ER model | 0.4 | Tier 3 |

Databases (DBMS) carries about **7.0 of the 85 technical marks** in a typical paper. Topics below are listed in the order given in the notes; use the table above to decide where to spend time first.

Legend: **Trap** = common mistake · **Tip** = shortcut. For practice, use `GATE_CS_Databases.pdf` (past questions + new questions with solutions). These notes were written from general knowledge, not checked line by line against a textbook; verify the less common facts.

# 1. SQL, Relational Algebra, Tuple Calculus

## Relational model

- Relation = set of tuples; keys: **superkey** (uniquely identifies tuples), **candidate key** (minimal superkey), primary key (chosen candidate key), **foreign key** (refers to a key of another relation; referential integrity). Number of superkeys of a relation with n attributes where one attribute is the only candidate key: $2^{n-1}$.
- **Relational algebra:** σ select, π project (removes duplicates), ∪, ∩, − (need union-compatible relations), × Cartesian product, ⋈ natural join, ⋈θ theta join, ρ rename, ÷ division ("for all" queries). Outer joins keep unmatched tuples with NULLs.
- Sizes: |R × S| = |R|·|S|; natural join between 0 and |R|·|S| tuples; if the join attribute is a key of S and a foreign key in R, the join size = |R|; selection ≤ |R|; projection ≤ |R|; union ≤ |R| + |S|.
- Relational algebra is the basis of query optimisation: push selections and projections down, choose join order. **Tuple relational calculus** {t | P(t)} and domain relational calculus are declarative equivalents (safe expressions have the same power as algebra).

## SQL essentials

- Order of evaluation: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY. WHERE filters rows before grouping; HAVING filters groups (and may use aggregates).
- Aggregates: COUNT(*), COUNT(col) (ignores NULLs), SUM, AVG, MIN, MAX. GROUP BY: every non-aggregated SELECT column must appear in GROUP BY.
- **NULL semantics:** comparisons with NULL give UNKNOWN; WHERE keeps only rows that evaluate to TRUE. `x NOT IN (subquery containing NULL)` is never TRUE. Use IS NULL / IS NOT NULL. COUNT(*) counts rows including NULLs; aggregates other than COUNT(*) ignore NULLs.
- Joins: INNER (matching rows), LEFT/RIGHT/FULL OUTER (keep unmatched). A natural join on R(x,y), S(y,z) with duplicates gives (matches per y value) products: y = a appearing 2× in each gives 4 rows.
- Subqueries: IN, EXISTS, ANY/ALL; correlated subqueries run per outer row. Set operations: UNION (removes duplicates), UNION ALL (keeps), INTERSECT, EXCEPT. DISTINCT removes duplicates.
- DDL: CREATE/ALTER/DROP; constraints: PRIMARY KEY, UNIQUE, NOT NULL, CHECK, FOREIGN KEY with ON DELETE CASCADE / SET NULL. DML: INSERT, UPDATE, DELETE. Views, triggers, indexes.
- "Highest/second-highest" queries: use a subquery with MAX or LIMIT/OFFSET; "for all" queries: NOT EXISTS (… NOT EXISTS …) or division.

> **Trap:** SQL tables are *bags* (duplicates allowed) unless DISTINCT is used; relational algebra works on sets. Counting rows after a join can differ for this reason.

## 2. Normalization & Functional Dependencies

- **FD** $X \to Y$: tuples agreeing on X agree on Y. Armstrong's axioms: reflexivity ($Y \subseteq X \Rightarrow X \to Y$), augmentation, transitivity; derived: union, decomposition, pseudo-transitivity.
- **Attribute closure $X^+$:** start with X; repeatedly add the right side of any FD whose left side is inside the current set. X is a superkey if $X^+$ = all attributes; a candidate key if also minimal.
- Finding candidate keys: attributes that never appear on any right side must be in every key; start with them and add attributes until the closure is complete. An attribute appearing only on right sides is in no key. For R(A,B,C,D,E) with A→B, B→C, C→D, D→E, E→A, each single attribute is a key (5 keys). Prime attribute = part of some candidate key.
- **Minimal (canonical) cover:** split right sides into single attributes, remove extraneous left attributes, remove redundant FDs. Two FD sets are equivalent if each is implied by the other (compare closures).
- **Normal forms:**
  • 1NF: atomic values.
  • 2NF: 1NF + no partial dependency of a non-prime attribute on part of a candidate key.
  • 3NF: for each FD $X \to A$, X is a superkey or A is prime (no transitive dependency of non-prime on a key).
  • BCNF: for each non-trivial FD $X \to Y$, X is a superkey. BCNF ⊂ 3NF ⊂ 2NF ⊂ 1NF.
  • 4NF: no non-trivial multivalued dependency unless on a superkey; 5NF: join dependencies.
- A relation whose every attribute is prime is always in 3NF. A relation with only two attributes is always in BCNF. If all keys are single attributes, the relation is in 2NF.
- **Decomposition properties: lossless-join** — for a binary decomposition $R \to R_1, R_2$ it holds iff $(R_1 \cap R_2) \to R_1$ or $(R_1 \cap R_2) \to R_2$; test general decompositions with the tableau (chase). **Dependency preserving** — every FD is implied by the union of projections. BCNF decomposition is always lossless but may lose dependencies; 3NF decomposition can always be both lossless and dependency preserving.
- To check a relation's highest normal form: list candidate keys, then test every FD against the BCNF and 3NF conditions in turn. Example: R(A,B,C,D), F = {AB → C, C → D, D → A}: keys AB, BC, BD; all attributes are prime ⇒ 3NF; C → D violates BCNF.

# 3. Transactions & Concurrency Control

- **ACID:** atomicity, consistency, isolation, durability. Transaction states: active, partially committed, committed, failed, aborted. Recovery uses logs (undo/redo, write-ahead logging, checkpoints).
- **Schedule:** an interleaving of operations (r, w, commit) of several transactions. Serial schedules are always consistent. A schedule is **conflict serializable** if it is conflict-equivalent to a serial schedule. Conflicting operations: same item, different transactions, at least one write (R-W, W-R, W-W).
- **Precedence graph test:** add an edge $T_i \to T_j$ for each conflict where $T_i$'s operation comes first. Conflict serializable $\Leftrightarrow$ the graph is acyclic; a topological order gives the equivalent serial schedule. **View serializable** is a broader class (blind writes); conflict $\subset$ view serializable. Testing view serializability is NP-complete.
- Number of possible serial schedules for n transactions: n!. Number of interleavings of transactions with $m_1$, $m_2$, … operations: $(\Sigma m_i)!/(\Pi m_i!)$.
- **Recoverability: recoverable** — if $T_j$ reads from $T_i$, then $T_i$ commits before $T_j$. **Cascadeless (ACA)** — transactions read only committed data. **Strict** — no read or write of a written item until that writer commits/aborts. Strict $\subset$ ACA $\subset$ recoverable. Dirty read, unrepeatable read, phantom, lost update anomalies; isolation levels (read uncommitted, read committed, repeatable read, serializable).
- **Two-phase locking (2PL):** growing phase (acquire locks) then shrinking phase (release only). 2PL guarantees conflict serializability but not freedom from deadlock or cascading rollbacks. **Strict 2PL** holds exclusive locks until commit (cascadeless); **rigorous 2PL** holds all locks until commit. Lock modes: shared (S), exclusive (X); compatibility: only S–S is compatible. The point where a transaction gets its last lock is its **lock point**; serial order = order of lock points.
- **Timestamp ordering:** each transaction gets a timestamp; operations that violate timestamp order cause abort and restart; no deadlock. Thomas' write rule ignores obsolete writes. Deadlock handling in locking: wait-for graph cycle detection; **wait-die** (older waits, younger dies) and **wound-wait** (older wounds younger).
- Multiple-granularity locking uses intention locks IS, IX, SIX.

# 4. Indexing, B/B+ Trees, File Organization

- **Blocking factor** bf = ⌊block size / record size⌋ records per block (no spanning). Number of blocks for a file = ⌈records / bf⌉. Unordered (heap) file: search n/2 blocks on average, insertion is cheap. Ordered file: binary search ⌈$\log_2$ b⌉ block accesses.
- Index types: **primary index** (on the ordered key field, sparse: one entry per block), **clustering index** (ordered non-key field), **secondary index** (on a non-ordering field, dense: one entry per record or per distinct value). Dense index: an entry for every record; sparse: one per block. Index entry size = key + block pointer. Index blocks = ⌈entries / entries per block⌉. Multi-level index: a sparse index on the index; levels until a single block remains.
- **Access cost:** single-level index: ⌈$\log_2$ (index blocks)⌉ + 1 (data block); multilevel: number of levels + 1; B+ tree: height + 1.
- **B-tree of order p:** every node has at most p children and p − 1 keys; internal nodes (except the root) have at least ⌈p/2⌉ children; leaves are on the same level; keys and data pointers in all nodes. **B+ tree:** data pointers only in leaves, leaves linked (fast range scans), internal nodes hold only keys, so a larger fan-out and fewer levels.
- **Order from block size:** internal node of a B+ tree: p·(pointer) + (p − 1)·(key) ≤ block size. Example: block 1024 B, key 8 B, pointer 6 B → 14p ≤ 1032 → p = 73. For a leaf: (p_leaf)(key + record pointer) + 1 next pointer ≤ block size.
- Maximum keys in a B+ tree of order p and height h: use fan-out p per level: leaves hold up to (p − 1) keys each (internal order p), so total keys ≈ $p^h$·(p − 1). Minimum keys: use the half-full minimum ⌈p/2⌉.
- Insertion: insert in the leaf; if it overflows, split and push the middle key up (B+ tree: copy the key up and keep it in the leaf). Deletion: remove, then borrow from or merge with a sibling if the node underflows. The height grows only when the root splits.
- **Hashing** (static/dynamic: extendible, linear). Cost comparisons: B+ tree is good for range queries; hash for equality.
- Query processing: cost of joins (nested loop: $b_R + b_R \cdot b_S$ block accesses; block nested loop; sort-merge; hash join), query optimisation (equivalence rules, cost estimates).

## 5. ER Model

- Components: **entity** (object), **attribute** (simple, composite, multivalued, derived, key attribute), **relationship** (association among entities), cardinality (1:1, 1:N, M:N), participation (total vs partial), **weak entity** (no key of its own; depends on an owner entity through an identifying relationship; has a partial key/discriminator; always total participation in the identifying relationship).
- Specialisation / generalisation (ISA), aggregation, disjoint vs overlapping, total vs partial.
- **Mapping ER → relational:**
  • strong entity → a relation with its attributes (key = primary key); multivalued attribute → a separate relation (entity key + value); composite → its components.
  • 1:1 → merge into either side's table (foreign key), preferably on the total-participation side. 1:N → foreign key on the N side. M:N → a new relation with both keys (plus relationship attributes); its primary key is the combination of both keys.
  • weak entity → a relation including the owner's key; primary key = owner key + partial key.
- **Minimum number of tables** = entities + M:N relationships + multivalued attributes (1:1 and 1:N relationships merge into existing tables). Example: three entities with one M:N, one 1:N and one 1:1 → 3 + 1 = 4 tables.
- Example derivations: an entity set with total participation in a 1:N relationship from the N side already carries the foreign key; a ternary relationship becomes a relation with the keys of all three entities.

## 6. Exam strategy & revision checklist

- Study order: normalization (keys, closure, normal forms) → SQL and relational algebra → transactions (serializability, 2PL) → indexing (B+ tree order, access counts) → ER.
- For SQL result questions, evaluate in the order FROM → WHERE → GROUP BY → HAVING → SELECT, and be careful with NULLs and duplicates.
- ☐ Compute closures and keys. ☐ Decide the highest normal form. ☐ Draw a precedence graph. ☐ Check strict 2PL conditions. ☐ Compute B+ tree order and index block counts. ☐ Map an ER diagram to the minimum number of tables. ☐ Redo `GATE_CS_Databases.pdf` past questions.